SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA04
COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.*

QUESTIONS: 5 **Total Marks:100**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Sathiksaakl 192524449. *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

90

Annexure A

Short Answer Test

Question 1 (5 Marks)
Suppose a modern Operating System supports multiple users and processes with shared resources like CPU, memory, and I/O devices.
(a) Classify different types of Operating Systems suitable for this scenario and justify your choice.
(b) Draw the structure of an Operating System showing how resources are managed.
(c) Write a sequence of system calls required to create and execute a new process in such an environment.

Question 2 (5 Marks) **(BL2)**
Consider an Operating System managing multiple files and devices for concurrent users.
(a) Categorize system calls used for file and device management.
(b) Illustrate how the OS handles file operations using a layered structure diagram.
(c) Write a pseudo-code or C-based system call sequence to open, read, and close a file.

Question 3 (5 Marks) **(BL3)**
Assume a real-time Operating System used in an embedded system controlling industrial machinery.
(a) Identify the type of OS and explain why it is suitable for real-time applications.
(b) Draw a diagram representing the interaction between hardware, kernel, and user processes.
(c) List and explain system calls required for process synchronization in this system.

(BL2)

Question 4 (5 Marks)
A distributed computing system consists of multiple interconnected computers sharing resources over a network.
(a) Classify the type of Operating System used and compare it with centralized OS.
(b) Draw the architecture of a distributed OS showing communication between nodes.
(c) Write the sequence of system calls involved in inter-process communication (IPC).

(BL3)

Question 5 (5 Marks)
Consider a scenario where an Operating System must manage process scheduling, memory allocation, and device communication efficiently.
(a) Explain the major functions of an OS in handling these tasks.
(b) Draw a block diagram representing OS components and their interactions.
(c) Write system call examples for memory allocation and device management.

(BL3)

RUBRICS FOR CALCULATION

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA0412
COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.*

QUESTIONS: 5 **Total Marks:100**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I SRITHIKA.K(192524449)___(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:SRITHIKA.K

Annexure A

Short Answer Test

Question 1 (5 Marks)

Suppose a modern Operating System supports multiple users and processes with shared resources like CPU, memory, and I/O devices.

(a) Classify different types of Operating Systems suitable for this scenario and justify your choice.

(b) Draw the structure of an Operating System showing how resources are managed.

(c) Write a sequence of system calls required to create and execute a new process in such an environment. **(BL2)**

Question 2 (5 Marks)

Consider an Operating System managing multiple files and devices for concurrent users.

(a) Categorize system calls used for file and device management.

(b) Illustrate how the OS handles file operations using a layered structure diagram.

(c) Write a pseudo-code or C-based system call sequence to open, read, and close a file. **(BL3)**

Question 3 (5 Marks)

Assume a real-time Operating System used in an embedded system controlling industrial machinery.

(a) Identify the type of OS and explain why it is suitable for real-time applications.

(b) Draw a diagram representing the interaction between hardware, kernel, and user processes.

(c) List and explain system calls required for process synchronization in this system. **(BL2)**

Question 4 (5 Marks)

A distributed computing system consists of multiple interconnected computers sharing resources over a network.

(a) Classify the type of Operating System used and compare it with centralized OS.

(b) Draw the architecture of a distributed OS showing communication between nodes.

(c) Write the sequence of system calls involved in inter-process communication (IPC). **(BL3)**

Question 5 (5 Marks)

Consider a scenario where an Operating System must manage process scheduling, memory allocation, and device communication efficiently.

(a) Explain the major functions of an OS in handling these tasks.

(b) Draw a block diagram representing OS components and their interactions.

(c) Write system call examples for memory allocation and device management. **(BL3)**

RUBRICS FOR CALCULATION

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

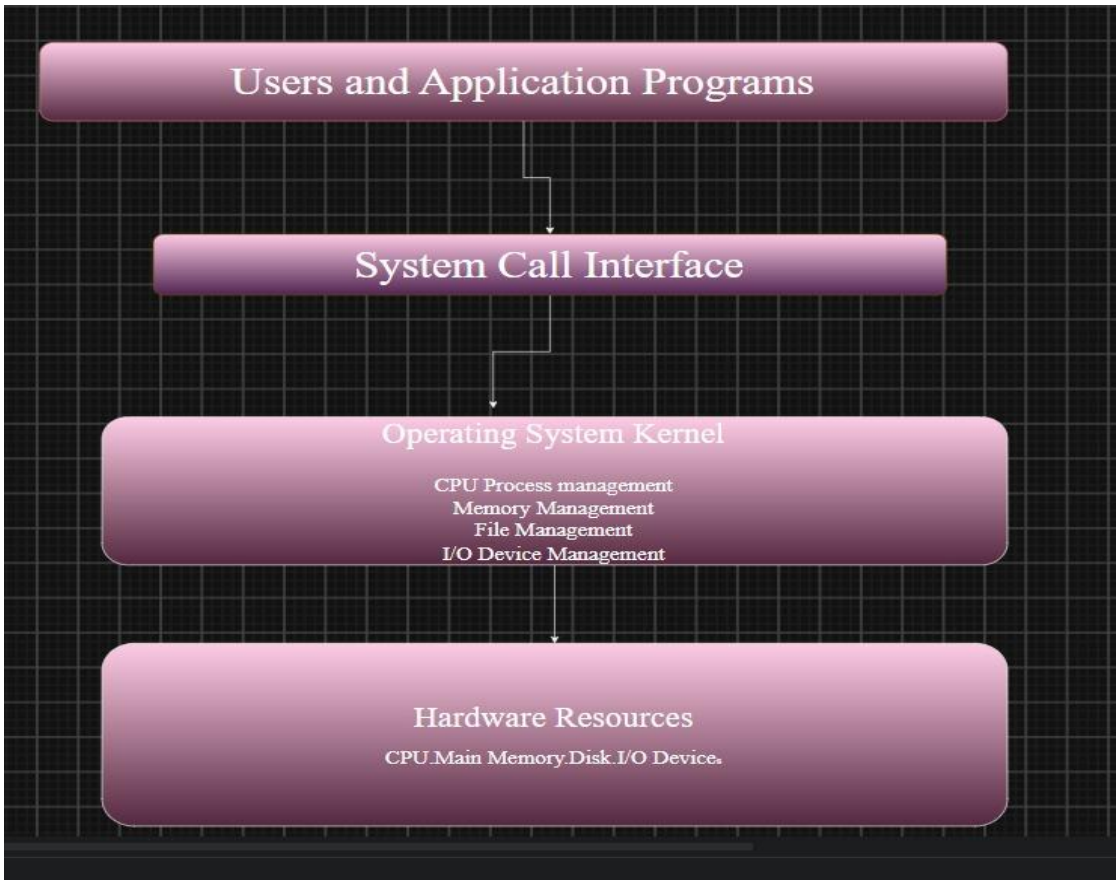
1.(a) Classify different types of Operating Systems and justify your choice

An Operating System (OS) manages computer resources such as CPU, memory, and I/O devices and allows multiple users and processes to work efficiently.

Type of OS	Description
Multi-user OS	Allows multiple users to access the system simultaneously.
Multitasking OS	Executes multiple processes by sharing CPU time.
Multiprocessing OS	Uses multiple CPUs or cores to execute processes.
Time-sharing OS	Allocates CPU time to each process for quick response.
Real-time OS	Provides responses within a specified time limit.

Justification: A multi-user, multitasking, time-sharing OS is most suitable for this scenario because it supports multiple users, manages multiple processes, and shares CPU, memory, and I/O resources efficiently. Multiprocessing can further improve performance when multiple CPU cores are available.

(b) Draw the structure of an Operating System showing resource management



Explanation:

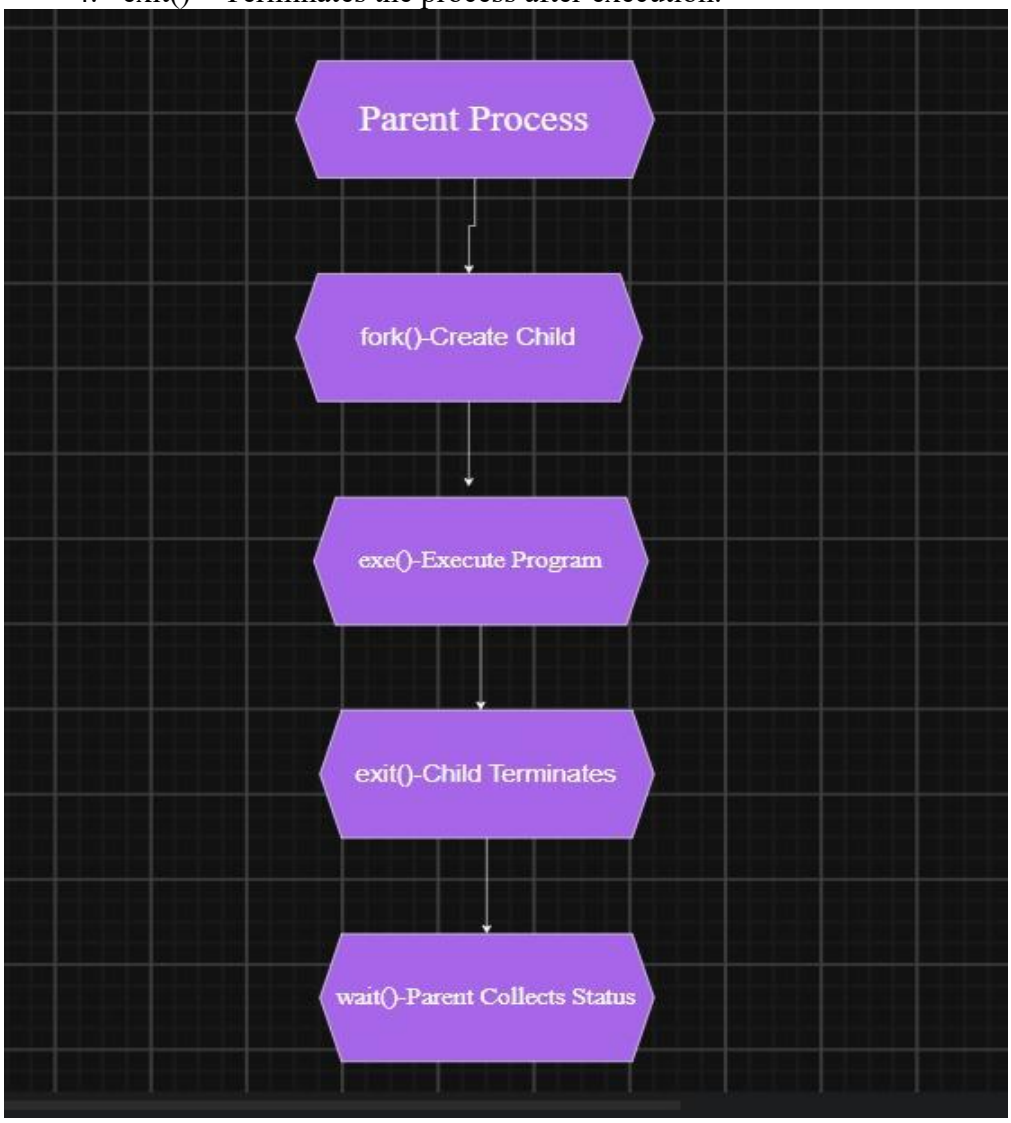
- Process management: Schedules processes and allocates CPU time.

- Memory management: Allocates and frees memory.
- File management: Organizes and accesses files

(c) Sequence of system calls to create and execute a new process

In a UNIX/Linux operating system, the following system calls are used:

1. fork() – Creates a new child process.
2. exec() – Loads and executes a new program in the child process.
3. wait() – Allows the parent process to wait for the child to finish.
4. exit() – Terminates the process after execution.



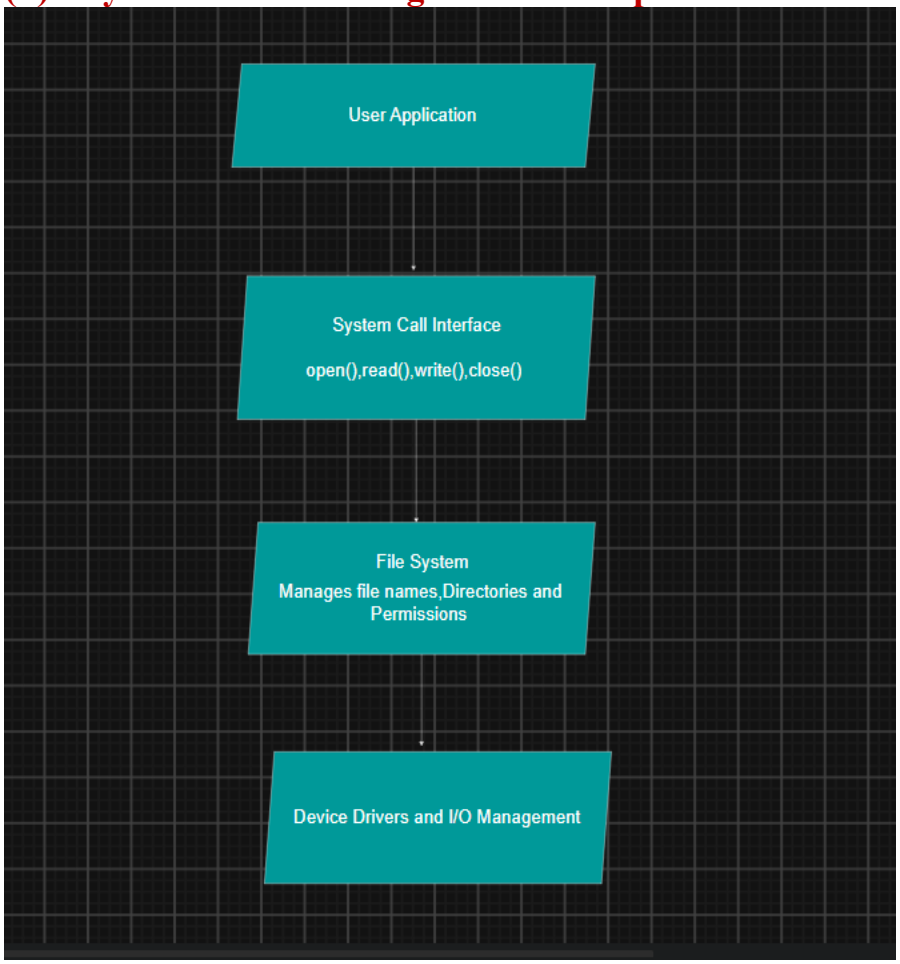
2. (a) Categorize system calls used for file and device management

System calls allow user programs to request file and device operations from the Operating System.

Category	System Calls	Purpose
File management	`open()`, `read()`, `write()`, `close()`	Opens, reads, writes, and closes files.

File information	`stat(), lseek()`	Gets file details and changes file position.
Device management	`ioctl(), open(), close()`	Controls and accesses devices.
Device I/O	`read(), write()`	Transfers data to and from devices.

(b) Layered structure diagram for file operations



(b) C program to open, read, and close a file

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char buffer[100];
    int n;

    fd = open("input.txt", O_RDONLY);

    if (fd < 0) {
        perror("open");
        return 1;
    }

    n = read(fd, buffer, sizeof(buffer) - 1);

    if (n < 0) {
        perror("read");
        close(fd);
        return 1;
    }

    buffer[n] = '\0';

    printf("File content: %s\n", buffer);

    close(fd);
```

```
    return 0;
}
```

3. (a) Identify the type of OS and explain its suitability

A Real-Time Operating System (RTOS) is used in embedded systems that control industrial machinery.

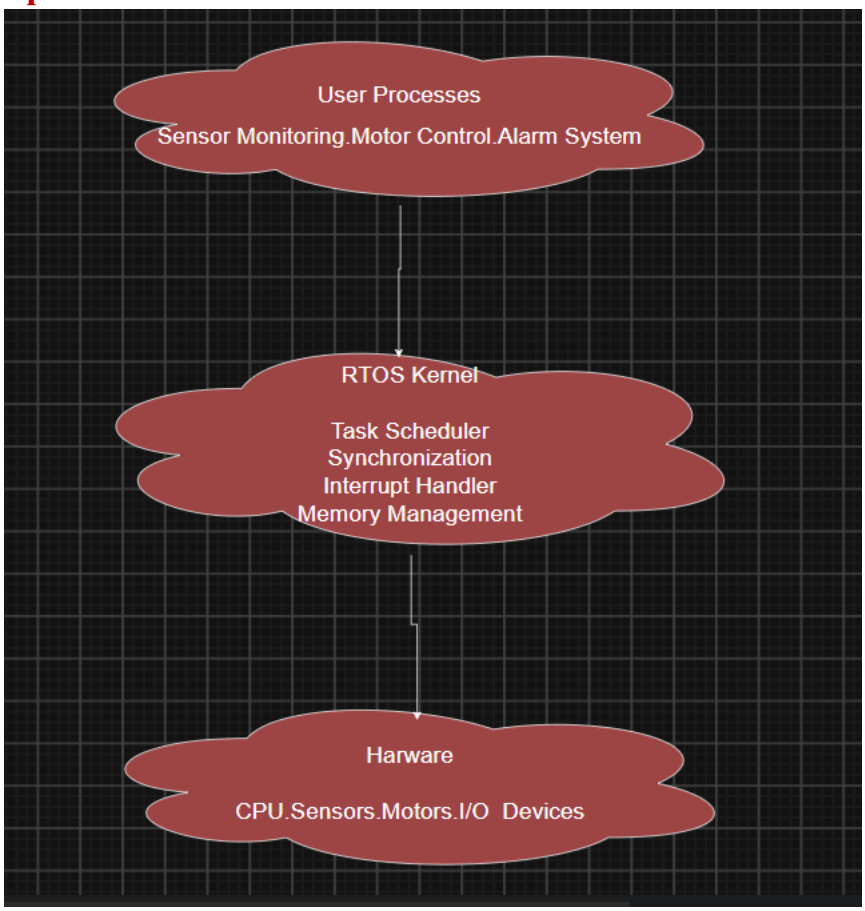
Types of RTOS:

1. **Hard Real-Time OS:** Tasks must be completed within strict deadlines. Missing a deadline may cause system failure.
2. **Soft Real-Time OS:** Small delays are acceptable, but performance may decrease.

Why RTOS is suitable:

- Provides fast and predictable responses.
- Schedules tasks according to priority.
- Supports process synchronization.
- Handles sensors, motors, and other hardware devices efficiently.
- Helps ensure industrial machinery operates safely and reliably.

(b) Diagram showing interaction between hardware, kernel, and user processes



Explanation: User processes request services from the RTOS kernel. The kernel schedules tasks, manages synchronization, and communicates with hardware through device drivers and

interrupt handlers.

(c) System calls required for process synchronization

System call / function	Purpose
`sem_wait()`	Waits for a semaphore before accessing a shared resource.
`sem_post()`	Signals a semaphore and allows another task to proceed.
System call / function	**Purpose**
`pthread_mutex_lock()`	Locks a shared resource to prevent simultaneous access.
`pthread_mutex_unlock()`	Unlocks the resource after use.
`pthread_cond_wait()`	Waits until a particular condition is satisfied.
`pthread_cond_signal()`	Signals a waiting thread when a condition is met.

These are POSIX synchronization functions commonly used in RTOS applications; their availability depends on the RTOS.

Result: An RTOS provides predictable task scheduling and synchronization, helping industrial machinery perform operations within required time limits.

4. (a) Classify the OS and compare it with a Centralized OS

Type of OS: Distributed Operating System

A Distributed Operating System manages multiple interconnected computers (nodes) and allows them to share resources and communicate through a network.

Feature	Distributed OS	Centralized OS
Structure	Multiple connected computers	One central computer
Resource sharing	Shared across nodes	Managed by one system
Reliability	Can continue if some nodes fail	Central failure may stop services
Scalability	Easy to add more nodes	Expansion is more limited
Communication	Uses network communication	Mainly uses local communication
Example	Distributed server and cluster systems	Traditional single-computer system

(b) Draw the architecture of a distributed OS showing communication between nodes



(c) Sequence of System Calls for Inter-Process Communication (IPC)

IPC allows processes to exchange data and communicate with each other.

Sequence using message queues:

1. msgget() – Creates or accesses a message queue.
2. msgsnd() – Sends a message to the queue.
3. msgrcv() – Receives a message from the queue.
4. msgctl() – Controls or removes the message queue.

Sequence:

msgget() → msgsnd() → msgrcv() → msgctl()

Other IPC system calls: pipe(), shmget(), shmat(), socket(), send(), and recv().

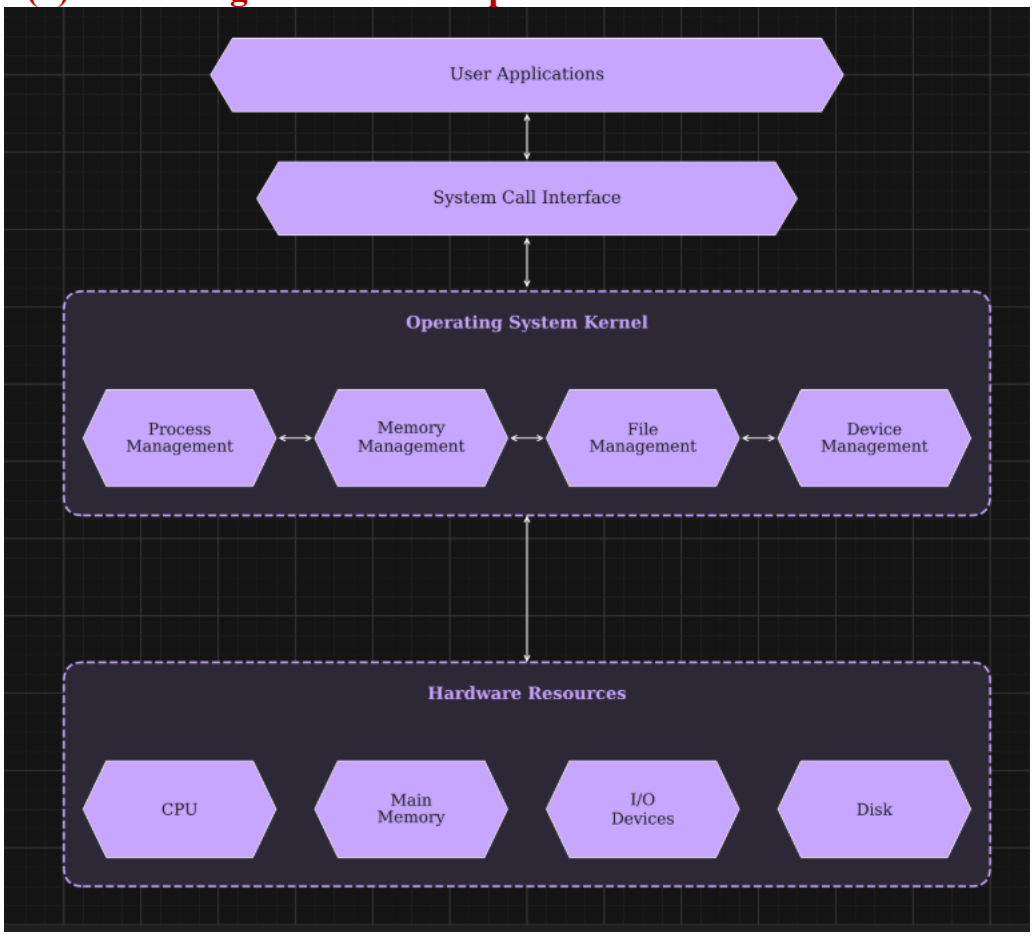
Result: IPC system calls allow processes to exchange information and coordinate their activities in a distributed system.

5. (a) Major functions of an OS

An Operating System manages hardware and software resources and provides services to application programs.

1. Process Scheduling: Allocates CPU time to processes and decides which process executes next.
2. Memory Management: Allocates and deallocates memory to programs and manages available memory.
3. Device Management: Controls I/O devices using device drivers and manages data transfer.
4. File Management: Creates, reads, writes, and deletes files.
5. Security Management: Protects system resources and prevents unauthorized access.

(b) Block diagram of OS components and their interactions



(c) System call examples for memory allocation and device management

Memory management:

- mmap() – Maps memory into a process's address space.
- munmap() – Releases a memory mapping.

Device management:

- open() – Opens a device file.
- read() – Reads data from a device.
- write() – Writes data to a device.
- close() – Closes the device file.

Result: The OS efficiently manages CPU scheduling, memory allocation, and device communication through its kernel components and system calls.